Ders 7 Genişletilmiş Notu: Matris Zinciri Çarpımı, Amortize Analiz ve Yineleme Bağıntısı Tekrarı

Ders 7 Genişletilmiş Notu: Matris Zinciri Çarpımı, Amortize Analiz ve Yineleme Bağıntısı Tekrarı

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Matris Zinciri Çarpımı Problemi
- 2. Özyinelemeli Bağıntının Çıkarılması
- 3. Kaç Farklı Parantezleme Var? Catalan Sayıları
- 4. Dört Matrisli Örnek ve Tekrar Eden Alt Problemler
- 5. Maliyet ve Parantez Tabloları
- 6. Matris Zincirinin Karmaşıklığı
- 7. Önceki Dersten Bir Soru: Anahtar Uzunluğu ve Çakışma
- 8. Amortize Analize Giriş
- 9. Toplam (Aggregate) Analiz
- 10. Muhasebe (Accounting) Yöntemi
- 11. Potansiyel (Potential) Yöntemi
- 12. Tekrar: Yineleme Bağıntılarını Geriye Doğru Yerine Koyma ile Çözmek
- Örnek Sorular

1. Matris Zinciri Çarpımı Problemi

Hoca derse, geçen hafta başlanan dinamik programlama konusuna bir örnek daha ekleyerek başladı. Dinamik programlamanın önemli bir konu olduğunu, kafada oturması için birkaç örnek görmek gerektiğini, memoization'ı nasıl uygulayacağınıza dair örnekler arttıkça uygulamanın kolaylaşacağını söyledi. Anlatımın tamamı Microsoft Whiteboard üzerinde yapıldı. Seçilen örnek, matris zinciri çarpımıdır (matrix chain multiplication).

Problem şudur: Elinizde birden fazla matris var ve bunları art arda çarpmak istiyorsunuz. Matris çarpımı zaten maliyetli bir iştir ve maliyet matrislerin boyutlarına bağlıdır. Zincir şeklinde çok sayıda matris çarpılacaksa maliyet daha da artar. Matris çarpımı birleşmeli olduğu için hangi çiftin önce çarpıldığı sonucu değiştirmez, ama yapılan çarpma sayısını değiştirir.

1.1 Üç Matrisli Örnek

Tahtadaki örnek (ekrandan, 00:10:00): A_1 2×3, A_2 3×4, A_3 4×2 boyutludur. Bu üç matris iki farklı sırayla çarpılabilir.

Birinci seçenek: $(A_1 A_2) A_3$**.** Önce A_1 ile A_2 çarpılır. 2×3 ile 3×4 boyutlu iki matrisin çarpımında, birinci matrisin her satırı ikinci matrisin her sütunuyla çarpılıp toplanır. Bu yüzden işlem sayısı $2 \cdot 3 \cdot 4 = 24$'tür. Sonuç 2×4 boyutludur. Bu sonuç 4×2 boyutlu A_3 ile çarpılınca $2 \cdot 4 \cdot 2 = 16$ çarpma daha yapılır. Toplam: $24 + 16 = 40$.

İkinci seçenek: $A_1 (A_2 A_3)$**.** Önce A_2 (3×4) ile A_3 (4×2) çarpılır: $3 \cdot 4 \cdot 2 = 24$ çarpma, sonuç 3×2. Sonra A_1 (2×3) bu sonuçla çarpılır: $2 \cdot 3 \cdot 2 = 12$ çarpma. Toplam: $24 + 12 = 36$.

Demek ki ikinci sıra daha kârlıdır. Algoritmanın amacı, bütün çarpım seçeneklerini değerlendirip en iyisini kullanıcıya döndürmektir. Üç matris varken bunu elle görmek kolaydır, ama on ya da yüz matris varken dinamik programlama mantıklı hâle gelir.

> **Ders sonrası not:** Altyazıda birinci seçenek anlatılırken iki yerde "A4 ile çarparım" geçiyor; örnekte yalnız üç matris vardır, kastedilen A_3'tür (ekrandan, 00:10:00).

2. Özyinelemeli Bağıntının Çıkarılması

Hoca bu küçük örnek üzerinden algoritmanın nasıl formüle edileceğini gösterdi. Önce boyutlara isim verdi: A_1 $d_0 \times d_1$, A_2 $d_1 \times d_2$, A_3 $d_2 \times d_3$ boyutludur. Ardışık iki matrisin iç boyutları aynı olmak zorunda olduğu için (A_1'in sütun sayısı A_2'nin satır sayısına eşit), bu ortak boyutlar aynı d ile gösterilir. Böylece A_i matrisi $d_{i-1} \times d_i$ boyutlu olur.

Sonra bir M tablosu tanımlandı: $M[i, j]$, i'inci matristen j'inci matrise kadar olan çarpımın en düşük maliyetidir. Tek bir matrisin çarpma maliyeti yoktur: $M[1,1] = M[2,2] = M[3,3] = 0$.

İki seçenek bu gösterimle şöyle yazılır:

- $(A_1A_2)A_3$: $M[1,2] + M[3,3] + d_0 \cdot d_2 \cdot d_3 = 24 + 0 + 16 = 40$. Burada $d_0d_2d_3$, 2×4'lük ara sonuç ile 4×2'lik A_3'ün çarpım maliyetidir.
- $A_1(A_2A_3)$: $M[1,1] + M[2,3] + d_0 \cdot d_1 \cdot d_3 = 0 + 24 + 12 = 36$ (ekrandan, 00:21:40).

Hoca bu iki ifadedeki tekrar eden kalıba dikkat çekti. Birinci indis i, son indis j her iki ifadede de aynı yerde duruyor. Ortadaki iki M teriminin indisleri birbirini izliyor: biri k'de bitiyor, öbürü $k+1$'de başlıyor. k, parantezin konduğu ayrım noktasıdır. d çarpımındaki indisler de $i-1$, k ve j'dir. Buradan genel bağıntı çıkar (ekrandan, 00:20:00):

$$M[i,j] = \begin{cases} 0 & i = j \\ \min\limits_{i \le k < j} \left(M[i,k] + M[k+1,j] + d_{i-1} \cdot d_k \cdot d_j \right) & i < j \end{cases}$$

k, i ile j arasında sınırlıdır. Örneğin $i = 1$, $j = 3$ ise k yalnız 1 ve 2 değerlerini alır. Formülün verdiği seçenekler arasından en küçüğü seçilir; böylece en iyi çözüm bulunur. Hoca bu bağıntının bundan sonra "ana formül" olacağını söyledi.

3. Kaç Farklı Parantezleme Var? Catalan Sayıları

Hoca, nasıl çıkarıldığını uzun olduğu için anlatmadan, n matris için kaç farklı seçenek denenmesi gerektiğini sordu. Bununla ilgili bir yineleme bağıntısı kurulursa karşınıza Catalan sayıları çıkar.

Dört matris için seçenekler tahtaya yazıldı (ekrandan, 00:21:40):

- $A_1(A_2(A_3A_4))$
- $A_1((A_2A_3)A_4)$
- $(A_1A_2)(A_3A_4)$
- $(A_1(A_2A_3))A_4$
- $((A_1A_2)A_3)A_4$

Tahtadaki formül, $m = n - 1$ olmak üzere:

$$\frac{(2m)!}{(m+1)!\,m!}$$

$n = 4$ için $m = 3$: $\frac{6!}{4!\,3!} = \frac{4! \cdot 5 \cdot 6}{4! \cdot 6} = 5$. Gerçekten beş seçenek vardır. Hoca boş vakitte Catalan sayılarına ve bu ilişkiye bakılmasını önerdi.

> **Ek bilgi (derste anlatılmadı):** Catalan sayıları $1, 1, 2, 5, 14, 42, \ldots$ diye gider ve büyük olasılıkla en çok bilinen kullanımı, n elemanlı bir çarpımın parantezleme sayısıdır. Bu sayılar n ile üstel hızda büyür; bu yüzden bütün parantezlemeleri tek tek denemek büyük zincirlerde uygulanabilir değildir.

4. Dört Matrisli Örnek ve Tekrar Eden Alt Problemler

Hoca formülü dört matrisli bir zincire uyguladı: A_1 $d_0 \times d_1$, A_2 $d_1 \times d_2$, A_3 $d_2 \times d_3$, A_4 $d_3 \times d_4$. Amaç $M[1,4]$'ü bulmaktır; k 1, 2 ve 3 değerlerini alır (ekrandan, 00:30:00):

- $k = 1$: $M[1,1] + M[2,4] + d_0 \cdot d_1 \cdot d_4$
- $k = 2$: $M[1,2] + M[3,4] + d_0 \cdot d_2 \cdot d_4$
- $k = 3$: $M[1,3] + M[4,4] + d_0 \cdot d_3 \cdot d_4$

Hoca $k = 1$ satırının anlamını açtı: $A_1(A_2A_3A_4)$ demektir. Önce A_2 ile A_4 arası çarpılır (maliyeti $M[2,4]$), sonra sonuç A_1 ile çarpılır. $A_2A_3A_4$ nasıl çarpılırsa çarpılsın sonuç $d_1 \times d_4$ boyutlu olur. A_1 $d_0 \times d_1$ olduğu için son çarpımın maliyeti $d_0 \cdot d_1 \cdot d_4$'tür. Hoca burada önce "artı" dedi, sonra bunların toplam değil çarpım olduğunu vurguladı. Bu açıklamayla, $k = 1$ seçeneğinde 2 ile 4 arasının tarandığını göstermek istedi.

$M[1,1]$ ve $M[4,4]$ zaten sıfırdır. Diğer terimler için yeniden hesap gerekir. Örneğin $M[2,4]$ için k 2 ve 3 olabilir:

- $k = 2$: $M[2,2] + M[3,4] + d_1 \cdot d_2 \cdot d_4$
- $k = 3$: $M[2,3] + M[4,4] + d_1 \cdot d_3 \cdot d_4$

Burada dikkat çeken şudur: $M[1,4]$'ü hesaplamak için $M[2,4]$ gerekir; $M[2,4]$ için $M[3,4]$ gerekir; $M[1,4]$'ün $k = 2$ satırında da yine $M[3,4]$ gerekir. Aynı şey birden fazla kez hesaplanır ve zincir büyükse bu hesaplar kolay değildir. Bu yüzden burada memoization mutlaka kullanılmalıdır. Hocaya göre memoization yapmadan problemi özyinelemeli çözmek size bir şey kazandırmaz.

5. Maliyet ve Parantez Tabloları

Bu problemde iki tabloya ihtiyaç vardır:

- M **tablosu (maliyet tablosu):** Hesaplanan maliyetleri tutar. Örneğin $M[2,4]$ bir kez hesaplanıp yazılır, sonra ihtiyaç olduğunda oradan okunur.
- P **tablosu (parantez tablosu):** Kullanıcıya yalnız en az çarpma sayısını vermek yetmez; parantezlerin nereye konacağını, yani hangi matrislerin önce çarpılacağını da söylemek gerekir. Bu tablo her hücre için en iyi sonucu veren ayrım noktası k'yi tutar.

Hoca tabloların hangi sırayla doldurulacağını anlattı. Köşegen sıfırdır, çünkü bir matrisi tek başına "çarpmanın" maliyeti yoktur. Önce aralarında yalnız bir fark olan küçük işlemler, yani $M[1,2]$, $M[2,3]$, $M[3,4]$ hesaplanır. Sonra bir üst çapraz ($M[1,3]$, $M[2,4]$), en son $M[1,4]$ doldurulur. P tablosu da aynı sırayla, her hesapta M ile birlikte doldurulur.

5.1 Sayısal Örnek

Tahtadaki boyutlar (ekrandan, 00:40:00): A_1 3×2, A_2 2×4, A_3 4×2, A_4 2×5. Yani $d = (d_0, d_1, d_2, d_3, d_4) = (3, 2, 4, 2, 5)$.

Hoca yalnız ilk hücreyi tahtada açtı. $M[1,2]$ için $1 \leq k < 2$, yani yalnız $k = 1$ vardır:

$$M[1,2] = M[1,1] + M[2,2] + d_0 \cdot d_1 \cdot d_2 = 0 + 0 + 3 \cdot 2 \cdot 4 = 24$$

Tek seçenek olduğu için karşılaştırılacak başka bir şey yoktur. Daha büyük aralıklarda k'nın her değeri için bir sonuç çıkar ve en düşüğü seçilir. 24'ü $k = 1$ verdiği için $P[1,2] = 1$ yazılır. Bunun anlamı: 1 ile 2 arasındaki çarpımda parantez 1. matristen hemen sonra kapanır.

Hoca diğer hücreleri öğrencilere bıraktı ve yalnız sonuçlarını yazdı. Ara adımlar aşağıda formülden kuruldu (ara adım eklendi):

- $M[2,3] = d_1d_2d_3 = 2 \cdot 4 \cdot 2 = 16$, $P[2,3] = 2$.
- $M[3,4] = d_2d_3d_4 = 4 \cdot 2 \cdot 5 = 40$, $P[3,4] = 3$.
- $M[1,3]$: $k = 1$: $0 + 16 + 3 \cdot 2 \cdot 2 = 28$; $k = 2$: $24 + 0 + 3 \cdot 4 \cdot 2 = 48$. En küçük 28, $P[1,3] = 1$.

- $M[2,4]$: $k=2$: $0+40+2\cdot4\cdot5=80$; $k=3$: $16+0+2\cdot2\cdot5=36$. En küçük 36, $P[2,4]=3$.
- $M[1,4]$: $k=1$: $0+36+3\cdot2\cdot5=66$; $k=2$: $24+40+3\cdot4\cdot5=124$; $k=3$: $28+0+3\cdot2\cdot5=58$. En küçük 58, $P[1,4]=3$.

Tahtadaki tablolar (ekrandan, 00:40:00; değerler python ile doğrulandı):

M	1	2	3	4
1	0	24	28	58
2		0	16	36
3			0	40
4				0

P	1	2	3	4
1		1	1	3
2			2	3
3				3

5.2 Parantezlemenin Okunması

P tablosu dıştan içe okunur. $P[1,4]=3$, A_1–A_4 arasında parantezin 3. matristen sonra konacağını söyler: $(A_1A_2A_3)A_4$. Sonra A_1–A_3 arası için $P[1,3]$'e bakılır: 1, yani parantez 1. matristen sonra: $A_1(A_2A_3)$. Doğrulamak için $P[2,3]$'e de bakılabilir: 2'den sonra parantez, ama zaten iki matris kalmıştır, daha fazla küçülmez. Sonuç (ekrandan, 00:40:00):

$$((A_1)(A_2A_3))A_4$$

Yani en iyi çözümde önce A_2 ile A_3 çarpılır, sonra sonuç A_1 ile, en son da A_4 ile çarpılır.

6. Matris Zincirinin Karmaşıklığı

Hoca, dinamik programlamayla çözülen matris zinciri çarpımının karmaşıklığının ne olduğunu sordu.

Soru: $m+n$ olabilir mi (matrislerin satır ve sütun sayıları)? **Cevap:** Hoca, tablolar zorunlu olarak kare olduğu için bunun $2n$, yani n anlamına geldiğini söyledi; bunun doğru cevap olmadığını belirtti. Bir başka öğrenci "kare olmaz mı?" diye sordu. Hoca iki cevaptan da birer parça alıp birleştireceğini söyledi.

Dinamik programlamadaki genel prensip hatırlatıldı: problemi alt problemlere ayırırsınız; kaç alt problem çözdüğünüze ve her biri için ne kadar vakit harcadığınıza bakarsınız. Tablodaki dolu hücre sayısı, çözülen alt problem sayısını gösterir. Dört matriste köşegenden başlayarak 4, 3, 2 ve 1 hücre doludur (ekrandan, 00:44:10):

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} \approx n^2$$

Köşegendeki sıfırlar aslında hesaplanmaz; bu değerden n çıkarılırsa gerçek değer bulunur, ama asimptotik gösterimde sınıf değişmez: n^2.

Her hücre için kaç işlem yapıldığına gelince: Her hücrede k, i ile j arasında değer alır. $i=1$, $j=4$ ise k 1, 2, 3 olur; $i=1$, $j=2$ ise yalnız 1 olur. Üst sınır hesaplandığı için bu sayı n alınır. Sonuç: n^2 hücrenin her biri için n işlem, toplam $O(n^3)$.

Soru: Beklediğimizden yüksek çıktı. **Cevap:** Hoca doğru olduğunu söyledi. Ama bunun dinamik programlamayla yapılmış hâl olduğunu, dinamik programlama olmadan ne olacağının da

düşünülmesini istedi. Memoization sayesinde her hücre bir kez hesaplanır; örneğin $M[2,4]$ gerektiğinde tablodan okunur. Bu yapılmasaydı maliyet daha da yüksek olurdu.

Soru: Bu yöntemin verimli olması için en az kaç matris gerekir? **Cevap:** Hoca, matrislerin boyutlarının, yani kaç hücreden oluştuklarının da önemli olduğunu söyledi. "Şu kadar işlem olursa verimli olur" diye bir araştırma yapmadığını, ama çok küçük zincirlerde verimli olmadığının bir gerçek olduğunu belirtti. Örneği, dinamik programlamanın mantığını iyi anlattığı için seçtiğini söyledi.

7. Önceki Dersten Bir Soru: Anahtar Uzunluğu ve Çakışma

Bir önceki derste bir öğrenci şunu sormuştu: hoca, anahtarın entropisinin yüksek olmasının ve daha fazla hane içermesinin çakışma (collision) riskini azaltacağını söylemişti; öğrenci ise böyle davranan bir hash fonksiyonunu kafasında kuramadığını söylemişti. Hoca bu soru üzerine küçük bir araştırma yaptığını anlattı. Evrensel (universal) aileye ait hash fonksiyonlarında anahtar boyutu ile çakışma riski arasındaki ödünleşimi (trade-off) inceleyen bir makale bulmuş. Makalede anahtardaki hane sayısı arttıkça çakışmanın azaldığını gösteren grafikler var ve bu bir hash ailesi için test edilmiş. Hoca makaleyi paylaşacağını söyledi. Derste anlatılan basit fonksiyonlarda bunun görülmesinin kolay olmadığını kabul edip küçük bir örnek kurdu.

Hash fonksiyonlarında bazen anahtarın tamamı bir anda hash'lenmez. Anahtar parçalara ayrılır, parçalar ayrı ayrı hash'lenir. Bu hash'ler art arda eklenebilir (concatenate), toplanıp tekrar hash'e verilebilir ya da biri seçilebilir.

Örnekte 16 bitlik bir anahtar iki 8 bitlik parçaya bölünür. Tahtadaki fonksiyonlar (ekrandan, 00:50:25):

$$h(key_1) = key_1 \bmod 7, \qquad h(key_2) = key_2 \bmod 11, \qquad h(key) = h(key_1) \,\|\, h(key_2)$$

Burada key_1 anlamsız (least significant) bitlerden oluşan alt parça, key_2 üst parçadır. Hoca birleştirme biçiminin önemli olmadığını, amacın fikri göstermek olduğunu söyledi.

Üç kişi, 16 hane kullanma şansları olduğu hâlde kolay olsun diye kısa anahtarlar seçmiş olsun (ekrandan, 00:50:25):

Anahtar	İkili gösterim	$key_1 \bmod 7$
2	00000000 00000010	2
9	00000000 00001001	2
16	00000000 00010000	2

Üst parça üçünde de sıfırdır, dolayısıyla $h(key_2)$ hep aynıdır. Alt parçanın hash'i de üçünde 2'dir. Bu üç anahtar çakışır (python ile doğrulandı). Üst haneler de doldurulsaydı her parça için ayrı bir değer hesaplanıp birleştirileceğinden çakışma bu kadar kolay olmazdı. Küçük, entropisi düşük, az haneli anahtarlar seçildiği için parçaların hash'leri hep aynı çıkmıştır.

8. Amortize Analize Giriş

Aradan sonra hoca amortize analize (amortized analysis) geçti. Şimdiye kadar çoğunlukla bir akışın en kötü durum senaryosu dikkate alındı; belki ortalama durum, ilk haftalarda da en iyi durum konuşuldu.

Amortize analizin motivasyonu şudur: Bir dizi işlem varsa, örneğin bir yığında (stack) sırayla push ve pop yapılıyorsa, her işlemin en kötü durumunu tek tek düşünmek bizi fazla kötümser (pesimistik) bir sonuca götürür. İşlem dizisinin tamamına bakmak daha gerçekçi bir yaklaşım sağlar.

Amortize analiz yapmanın birden fazla yolu vardır. Hoca üç yöntemi kısaca anlatacağını ve her biri için iki temel örnek (yığın işlemleri ve ikili sayaç) çözeceğini söyledi (ekrandan, 01:10:00).

Ek bilgi (derste anlatılmadı): Amortize analiz ortalama durum analizinden farklıdır: olasılık ya da girdi dağılımı varsaymaz. Büyük olasılıkla en bilinen kullanım alanları dinamik diziler (dolunca boyutu ikiye katlanan tablolar) ve birleşim-bul (union-find) gibi veri yapılarıdır; bunlarda tek bir işlem pahalı olsa da dizi boyunca işlem başına maliyet düşük kalır.

9. Toplam (Aggregate) Analiz

Birinci yöntem toplam analizdir (aggregate analysis). n işlemden oluşan bir dizi varsa, bu n işlem için en kötü durumda toplam maliyet $T(n)$ bulunur. Yani burada da bir en kötü durum vardır: sürekli pop yapmak mı, sürekli push yapmak mı, yoksa bunların bir karışımı mı sistemi daha çok zorlar, ona bakılır. Ama birbiriyle bağlantılı işlemler için sistemin tamamı düşünülmek zorundadır. Sonra toplam maliyet işlem sayısına bölünerek işlem başına ortalama (amortize) maliyet bulunur: $T(n)/n$ (ekrandan, 01:15:50).

9.1 Örnek 1: Yığın İşlemleri

Yığında push ve pop yapılabilir. Push için en basit hâliyle şu yazılır (ekrandan, 01:15:50):

```
PUSH(x):
    A[top] = x
    top++
```

Push da pop da tek adımda yapılır, $O(1)$'dir. n işlemin hepsi push ya da pop ise karmaşıklık $\Theta(n)$ olur.

Şimdi bir işlem daha eklensin: birden fazla elemanı çekebilen MULTIPOP. Bu işlem parametre almak zorundadır: S yığınından k eleman çeker. Tahtadaki sözde kod (ekrandan, 01:15:50):

```
MULTIPOP(S, k):
    while not STACK-EMPTY(S) and k > 0
        POP(S)
        k = k - 1
```

Yığın boş değilken ve k sıfırdan büyükken pop yapılır; k sıfıra inince yeterince eleman silinmiş olur.

MULTIPOP'un en kötü durumu, yığın doluyken (yığın boyutu n) hepsinin çekilmesidir: $O(n)$. İşlem kümesi PUSH, POP ve MULTIPOP olduğunda maliyeti en çok artıracak dizi, hepsi MULTIPOP olan bir dizidir; çünkü PUSH ve POP birer adımdır. n işlemin hepsi MULTIPOP ve her biri $O(n)$ ise sonuç $O(n^2)$ çıkar. Normal en kötü durum analizi bunu verir.

Ama gerçekte $O(n^2)$'ye ulaşılamaz. Boş bir yığından sürekli eleman çekemezsiniz; MULTIPOP mutlaka PUSH ile birlikte kullanılmak zorundadır. Hiçbir zaman push yapmadan pop yapılamayacağı için, n işlemden oluşan bir dizide çekilebilecek eleman sayısı yapılmış push sayısını aşamaz. Toplam maliyet en fazla $O(n)$'dir. Hoca "birbiriyle bağlantılı işlemleri düşünmeliyiz" derken bunu kastettiğini söyledi.

Ortalama maliyet: toplam maliyet $O(n)$, işlem sayısı n; işlem başına $O(n)/n = O(1)$. PUSH, POP ve MULTIPOP için ortalama maliyet yine sabittir. Toplam analizle gerçeğe daha yakın bir sınır elde edilmiş olur.

Soru: İşlem dizisine PUSH, POP ve MULTIPOP'un yanında bir de aynı anda k eleman ekleyen MULTIPUSH eklense, amortize ortalama yine $O(1)$ olur mu? **Cevap:** Olmaz. Artık tek işlemde birden fazla eleman eklenebilir. MULTIPOP'ta eleman bir kez çekildikten sonra push yapılmadan tekrar çekilemez, ama MULTIPUSH her seferinde k eleman koyabilir. En kötü durumda her işlemin maliyeti $O(k)$ olur. Yani sistemdeki işlem havuzu amortize maliyeti doğrudan etkiler.

9.2 Örnek 2: İkili Sayacı Artırma

İkinci örnek ikili sayaç artırmadır (incrementing a binary counter). Sayaç, her elemanı bir bit tutan bir dizidir. Örneğin 8 bitlik bir sayaç 8 elemanlı bir dizidir; en anlamlı bit (most significant

bit) $A[7]$'de, en anlamsız bit $A[0]$'dadır ve her eleman 0 ya da 1'dir. Her artırmadan sonra ilgili bitler güncellenir.

Artırma şöyle çalışır: Sayaç sıfırdan başlar. En sağdaki bit 0 ise 1 yapılır. 1 ise elde (carry) oluşur: yan yana duran 1'lerin hepsi 0'a çevrilir ve onlardan sonraki ilk 0, 1 yapılır. Tahtadaki sözde kod (ekrandan, 01:21:30):

```
INCREMENT(A):
    i = 0
    while i < A.length and A[i] == 1
        A[i] = 0
        i = i + 1
    if i < A.length
        A[i] = 1
```

Artırmanın maliyeti, değiştirilen (flip edilen, 0'dan 1'e ya da 1'den 0'a çevrilen) bit sayısıdır. Hoca ekranda 0'dan 16'ya kadar sayaç değerlerini ve birikimli toplam maliyeti gösteren bir tablo paylaştı (ekrandan, 01:20:00). Tablodaki toplam maliyetler şunlardır:

Sayaç değeri	0	1	2	3	4	5	6	7	8
Toplam maliyet	0	1	3	4	7	8	10	11	15

Sayaç değeri	9	10	11	12	13	14	15	16
Toplam maliyet	16	18	19	22	23	25	26	31

Örneğin 0'dan 1'e geçerken yalnız bir flip vardır, maliyet 1. 1'den 2'ye geçerken $A[0]$ sıfırlanır ve $A[1]$ 1 olur, iki flip; toplam 3. 3'ten 4'e geçerken üç flip; toplam 7. Flip sayısı toplam maliyeti verir; o hâlde karmaşıklığı bulmak için kaç flip yapıldığı formüle edilmelidir (değerler python ile doğrulandı).

Her bitin kaç kez değiştiğine bakılır. 16 artırmada $A[1]$ 8 kez, $A[2]$ 4 kez, $A[3]$ 2 kez değişir (ekrandan, 01:25:10). Genel olarak n artırmada $A[i]$ biti $\lfloor n/2^i \rfloor$ kez değişir. $A[0]$ her artırmada değişir. Toplam flip sayısı (ekrandan, 01:25:10):

$$\sum_{i=0}^{k-1} \left\lfloor \frac{n}{2^i} \right\rfloor < n \sum_{i=0}^{\infty} \frac{1}{2^i} = 2n$$

n, i'den bağımsız olduğu için toplamın dışına çıkarılır; kalan geometrik seri 2'ye eşittir. Toplam maliyet $2n$'den küçük olduğuna göre $O(n)$'dir. İşlem başına ortalama maliyet $O(1)$'dir.

Soru: Sayaçta artırmanın (INCREMENT) yanında bir de her seferinde bir azaltan DECREMENT olsaydı amortize maliyet ne olurdu? **Cevap:** En kötü durumu verecek dizi, en çok flip yaptıran durumda bir artırıp bir azaltmaktır. Örneğin en anlamlı bit 1, geri kalan $k-1$ bit 0 olsun. Azaltılınca o bit 0 olur, diğerlerinin hepsi 1'e döner: k flip. Sonra artırılınca sayı eski hâline döner: yine k flip. Her işlemde k flip yapılır; n işlem için toplam $O(nk)$ olur.

10. Muhasebe (Accounting) Yöntemi

İkinci yöntem muhasebe yöntemidir (accounting method). Çalışma mantığı farklı işlere farklı maliyetler atamaktır. Çok pahalı olmayan işlere fazladan maliyet atanır ve bu fazlalık diğer işlerin maliyetinden düşülür. Hoca bunu bir yerde para biriktirmeye benzetti.

10.1 Tanımlar

Tahtadaki tanımlar (ekrandan, 01:33:30):

- **Amortize maliyet (amortized cost):** "Amount we charge an operation", yani bir işleme biçtiğimiz maliyet. $\widehat{c}_i$, i'inci işlemin amortize maliyetidir. Bunu analizi yapan kişi belirler.

- **Gerçek maliyet:** c_i, i'inci işlemin gerçek maliyetidir.

Şu eşitsizlik sağlanırsa amortize maliyetler toplamı bir üst sınır (upper bound) verir:

$$\sum_{i=1}^{n} \hat{c}_i \geq \sum_{i=1}^{n} c_i$$

Bunun için toplam kredinin (total credit) negatif olmaması gerekir:

$$\sum_{i=1}^{n} \hat{c}_i - \sum_{i=1}^{n} c_i \geq 0$$

Bu, aynı eşitsizliğin terimleri yer değiştirmiş hâlidir. Amortize maliyetler belirlenirken bu koşula uyulup uyulmadığı mutlaka kontrol edilmelidir.

10.2 Örnek 1: Yığın İşlemleri

Gerçek ve amortize maliyetler (ekrandan, 01:33:30):

İşlem	Gerçek maliyet	Amortize maliyet
PUSH	1	2
POP	1	0
MULTIPOP	$\min(s, k)$	0

MULTIPOP'un gerçek maliyeti hangi argümanla çağrıldığına bağlıdır. Yığın boyutu s ve çekilmek istenen eleman sayısı k'dir; olmayan eleman çekilemeyeceği için maliyet bunların küçüğüdür.

Soru şudur: PUSH'a olması gerekenden fazla (2), POP ve MULTIPOP'a olması gerekenden az (0) değer atanırsa sistem dengede kalır mı, eşitsizlik hâlâ sağlanır mı?

Hoca bunu restoranlardaki tabak yığınlarıyla anlattı: tabaklar tek tek üst üste konur ve hep en üstten çekilir. PUSH için tabak koyma maliyeti 2 lira olarak belirlenmiştir. Tabak konurken bu 2 liranın 1'i koyma işinin gerçek maliyetine harcanır, 1 lirası tabağın içinde kalır. Pop yapılırken gerçek maliyet 1'dir, ama ekstra para ödemeye gerek yoktur: tabağın içinde birikmiş 1 lira vardır. Bu yüzden POP'un amortize maliyeti 0'dır. Hoca önce bunu yanlış anlattığını söyleyip bu şekilde düzeltti.

Burada yapılan şey, birbirini amortize edecek işlemleri bulmak ve buna uygun maliyetleri belirlemektir. Yığında negatif sayıda tabak olamayacağı, push yapmadan pop yapılamayacağı ve her push yanında bir ekstra kredi getirdiği için kredi hiçbir zaman negatife düşmez. Böylece koşul korunur.

Sonuç: PUSH'a fazla maliyet yüklenip POP için hiç ödeme yapılmadığı için maliyet hesabı yalnız PUSH'un maliyetine bakılarak çözülebilir hâle gelir. n işlem için toplam amortize maliyet $O(n)$'dir. Bu, birinci yöntemin verdiği cevapla aynıdır, yalnız farklı bir yoldan bulunmuştur.

Soru: MULTIPOP'un amortize maliyeti neden 0 oldu, gerçek maliyeti $\min(s, k)$ değil miydi?
Cevap: $\min(s, k)$ gerçek maliyettir. MULTIPOP yapısı gereği birden fazla POP işlemi demektir (içinde üç, beş, kaç tane olursa). POP'un amortize maliyeti nasıl 0 yapıldıysa MULTIPOP'unki de 0 yapıldı. PUSH'a 1 yerine 2 biçildiği için oradan gelen ekstra kredi hem POP'u hem MULTIPOP'u karşılamak için kullanılır; tabak örneğindeki gibi her çekilen tabağın içinde kendi lirası vardır.

10.3 Örnek 2: İkili Sayaç

İkili sayaçta karmaşıklığın flip sayısıyla ilişkili olduğu görülmüştü. Gerçek maliyetler (ekrandan, 01:40:00): bir biti 1 yapmanın (set a bit to 1) maliyeti 1, bir biti 0 yapmanın (set a bit to 0) maliyeti 1'dir.

Amortize maliyetler: biti 1 yapmaya 2, 0 yapmaya 0 verilir. Tabak örneğindeki gibi, her 1 yapılan bite 2 maliyet verilir; gerçek maliyet 1 olduğu için sayaçtaki her 1'in üzerinde bir ekstra kredi durur. O bit daha sonra 0'a çevrilirken bu kredi kullanılır. Bu yüzden 1'i 0'a çevirmek için ekstra maliyet yüklemek gerekmez. Hoca, temelde iki maliyetin toplanıp 2 yapıldığını, mantığın bu olduğunu söyledi.

Artırmanın amortize maliyetini hesaplamak için sözde koda dönüldü: döngüde 1'ler sıfırlanır (bunlar biriken kredilerle ödenir), sonunda en fazla bir kez 0, 1'e çevrilir ve bunun amortize maliyeti 2'dir. Sayaçtaki 1'lerin sayısı hiçbir zaman negatife düşmez; bu yüzden kredi miktarı da hep negatif olmayan kalır ve koşul sağlanır.

n artırma işleminin toplam amortize maliyeti $O(n)$'dir; tek bir işlem için n'ye bölününce $O(1)$ çıkar.

> **Ders sonrası not:** Hoca bir yerde her artırmanın amortize maliyetinin "bir" olduğunu söyledi. Biçilen maliyetlerle (1 yapmak 2, 0 yapmak 0) bir artırmanın amortize maliyeti en fazla 2'dir, yani $O(1)$; sonuç değişmez.
>
> **Hocanın vurgusu:** Bu problemleri çözmenin kilit noktası, maliyetin hangi büyüklük üzerinden hesaplanacağını belirlemektir. İkili sayaçta bunun 1'ler (flip'ler) üzerinden, yığında yığındaki eleman sayısı üzerinden yapılması gerektiğini anlamak gerekir.

11. Potansiyel (Potential) Yöntemi

Son aradan sonra hoca üçüncü ve son yöntem olan potansiyel yöntemine (potential method) geçti. Biraz daha matematiksel bir görünümü olduğunu, kulakta kalsın diye anlattığını söyledi.

Potansiyel yöntemi ilhamını fizikten alır: potansiyel enerji biriktirmek gibi düşünülür. Önceden ödeme yapılmışsa potansiyel enerji yüksek tutulur; önceden ödenmiş işlemler, ödenmemiş olanları yine benzer mantıkla karşılar.

11.1 Gösterim

n işlem bir veri yapısı üzerinde yapılır (ekrandan, 02:07:15):

- D_0: veri yapısının başlangıç hâli.
- c_i: i'inci işlemin gerçek maliyeti ($i = 1, \dots, n$).
- D_i: i'inci işlem yapıldıktan sonra veri yapısının aldığı yeni hâl.
- Φ: potansiyel fonksiyonu. Veri yapısının bir hâli için gerçek bir sayı üretir. $\Phi(D_i)$, D_i hâlindeyken önceden yapılmış ödemeleri, yani potansiyeli gösterir.
- $\hat{c}_i$: i'inci işlemin amortize maliyeti.

Amortize maliyet, gerçek maliyet ile i'inci işlemden kaynaklanan potansiyel değişiminin toplamıdır:

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

11.2 Teleskopik Toplam

n işlem için toplandığında (ekrandan, 02:07:15):

$$\sum_{i=1}^{n} \hat{c}_i = \sum_{i=1}^{n} (c_i + \Phi(D_i) - \Phi(D_{i-1})) = \sum_{i=1}^{n} c_i + [\Phi(D_1) - \Phi(D_0) + \Phi(D_2) - \Phi(D_1) + \cdots + \Phi(D_n) - \Phi(D_{n-1})]$$

Köşeli paranteze dikkat edilirse birbirini götüren terimler vardır: $+\Phi(D_1)$ ile $-\Phi(D_1)$, $+\Phi(D_2)$ ile $-\Phi(D_2)$, ... Geriye yalnız ilk ve son terim kalır:

$$\sum_{i=1}^{n} \hat{c}_i = \sum_{i=1}^{n} c_i + \Phi(D_n) - \Phi(D_0)$$

Yani toplam amortize maliyet aradaki değişimlere bağlı değildir, yalnız son ve ilk değere bağlıdır: veri yapısının son hâli ile ilk hâli arasındaki potansiyel farkı ile gerçek maliyetlerin toplamıdır.

11.3 Koşul

Φ öyle tanımlanabilirse ki (if we can define Φ so that) $\Phi(D_n) \geq \Phi(D_0)$ olsun, toplamdaki fark negatif olmaz ve

$$\sum_{i=1}^{n} \hat{c}_i \geq \sum_{i=1}^{n} c_i$$

olur. Amortize toplam gerçek maliyetten büyük ya da ona eşittir; yani bir üst sınırdır. Uygulamada genellikle veri yapısının ilk hâlinin potansiyeli sıfır alınır, $\Phi(D_0) = 0$, ve bütün ara değerlerde potansiyelin negatife düşmediği sağlanır: her i için $\Phi(D_i) \geq 0$. İşlemler bu koşulu sağlayacak şekilde tanımlanır.

> **Ders sonrası not:** Tahtada (02:07:15) son eşitsizlik "$>$" ile yazıldı; doğrusu "$\geq$"dir. Potansiyel farkı sıfır olabilir.

11.4 Örnek 1: Yığın İşlemleri

Yığın için potansiyel fonksiyonu yığındaki eleman sayısıdır (number of objects in the stack) (ekrandan, 02:10:00). Boş yığınla başlanır, $\Phi(D_0) = 0$. Yığındaki eleman sayısı hiçbir zaman negatife düşemez, $\Phi(D_i) \geq 0$; dolayısıyla $\Phi(D_i) \geq \Phi(D_0)$ koşulu sağlanır. Geriye her işlem için $\Phi(D_i) - \Phi(D_{i-1})$ farkını hesaplamak kalır.

PUSH: i'inci işlemden önce yığında s eleman olsun. İşlemden sonra $s+1$ eleman vardır. Fark: $(s+1) - s = 1$. Gerçek maliyet 1 olduğu için (ekrandan, 02:14:20):

$$\hat{c}_i = 1 + 1 = 2 \;\Rightarrow\; O(1)$$

Tek bir sabit sayı olduğu için asimptotik gösterimde $O(1)$'e denk gelir.

MULTIPOP: k verilen argüman, s yığındaki eleman sayısıdır; bunlardan hangisi küçükse o kadar eleman çekilir. Buna $k' = \min(k, s)$ denir; k' yığından çekilen eleman sayısıdır. D_{i-1}'de D_i'den k' kadar fazla eleman vardır, fark $-k'$'dir. Gerçek maliyet k' olduğu için (ekrandan, 02:14:20):

$$\hat{c}_i = k' - k' = 0 \;\Rightarrow\; O(1)$$

POP: Push'un tersine eleman sayısı bir azalır; fark -1'dir:

$$\hat{c}_i = 1 - 1 = 0 \;\Rightarrow\; O(1)$$

PUSH, POP ve MULTIPOP'un hepsi amortize $O(1)$'dir. n elemanlık bir işlem dizisinin toplam amortize maliyeti $O(n)$'dir.

Hoca iki yöntem arasındaki farkı şöyle özetledi: muhasebe yönteminde push'a ve pop'a tek tek maliyet atanmıştı; burada yığının tamamı ve biriktirdiği enerji düşünülür, durumlar arasındaki farklar hesaba katılır.

Soru: MULTIPOP'ta fark nasıl $-k$ oldu? **Cevap:** D_{i-1} durumundaki eleman sayısı D_i'dekinden k' fazladır, çünkü pop yapılmıştır. Örneğin önceki durumda $m + k'$ eleman varsa ve k' tanesi çekilirse m eleman kalır. Fark $m - (m + k') = -k'$'dir. Bu fazlalık azalma yönündedir, çünkü eleman çekilmiştir. Dersin sonunda bir öğrenci aynı yeri tekrar sordu ve farkın $-2k'$ olup olmadığını sordu; hoca aynı açıklamayı yineleyip farkın $-k'$ olduğunu, gerçek maliyet de k' olduğu için amortize maliyetin 0 çıktığını söyledi.

11.5 Örnek 2: İkili Sayacı Artırma

Yığında potansiyel nesnelerin sayısıydı. İkili sayaçta da 1'lerin sayısı potansiyel olarak alınabilir: Φ = sayaç dizisindeki 1'lerin sayısı (number of 1's in the counter array) (ekrandan, 02:20:00).

Gerçek maliyet: İşlem havuzunda yalnız artırma vardır. i'inci işlemin gerçek maliyeti $t_i + 1$'dir; t_i elde (carry) bitlerinin sayısıdır. Örneğin tahtada 00111 artırılınca 01000 olur: üç 1 sıfırlanır ($t_i = 3$) ve sonunda bir 0, 1 yapılır. Elde biti sayısı kadar flip, artı sondaki bir flip.

Potansiyel farkı: i'inci adımda t_i adet 1, 0 olur; demek ki $i-1$'inci adımda bu t_i adet 1 vardı ve kayboldu: $-t_i$. Sonra bir adet 0, 1 yapılır: $+1$. Fark $1 - t_i$'dir. O hâlde (ekrandan, 02:20:00):

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = (t_i + 1) + (1 - t_i) = 2$$

t_i'ler birbirini götürür. Sayaç sıfırdan başladığı için başlangıç potansiyeli sıfırdır ve 1'lerin sayısı hiçbir zaman negatife inmez; koşullar sağlanır. Bir işlem için amortize maliyet $O(1)$, n işlem için toplam $O(n)$'dir.

> **Ders sonrası not:** Hoca burada "total kostu N operasyon için O1" dedi; doğrusu işlem başına $O(1)$, n işlem için toplam $O(n)$'dir.
>
> **Ek bilgi (derste anlatılmadı):** Üç yöntem aynı problem için aynı sınırı verir; fark yalnız hesabın nasıl düzenlendiğindedir. Toplam analiz bütün diziyi doğrudan sayar, muhasebe yöntemi krediyi tek tek elemanlara (tabaklara, 1 bitlerine) yazar, potansiyel yöntemi ise krediyi veri yapısının durumunun bir fonksiyonu olarak ifade eder. Büyük olasılıkla karmaşık veri yapılarında potansiyel yöntemi daha çok tercih edilir, çünkü krediyi ayrı ayrı izlemek gerekmez.

12. Tekrar: Yineleme Bağıntılarını Geriye Doğru Yerine Koyma ile Çözmek

Kalan vakitte hoca, öğrencilerin tekrar isteği üzerine hazırladığı hatırlatma sorularından birkaçını çözdü. Yineleme bağıntıları (recurrence relation) karmaşıklık hakkında çıkarım yapmak için çözülür. Bunun birden fazla yolu vardı: ana teorem (master theorem), yerine koyma (substitution), ağaç (özyineleme ağacı) ve tümevarım (induction). Hoca tümevarımla ilgili gelen bir soru için de örnek hazırladığını söyledi. Bu derste en basiti olan geriye doğru yerine koymayla (backward substitution) başlandı.

12.1 Örnek: $x(n) = x(n-1) + 5$

Bağıntı: $x(n) = x(n-1) + 5$, $n > 1$, $x(1) = 0$.

Yöntem: bir adım geriye gidilir. $x(n-1) = x(n-2)+5$ olduğu için yukarıdaki ifadede $x(n-1)$ yerine bu konur:

$$x(n) = x(n-2) + 5 + 5$$

Sonra $x(n-2) = x(n-3) + 5$ yerine konur:

$$x(n) = x(n-3) + 3 \cdot 5$$

Bu birkaç adım, i'ye bağlı genel bir kalıp görülene kadar sürdürülür:

$$x(n) = x(n-i) + 5i$$

Sonra başlangıç koşuluna bakılarak i ortadan kaldırılır. Elimizde $x(1)$ verilmiştir, bu yüzden $n - i = 1$ olmalıdır, yani $i = n - 1$:

$$x(n) = x(1) + 5(n-1) = 0 + 5n - 5 = 5n - 5$$

Böylece x'e bağlı olmadan, yalnız n'e bağlı bir sonuç elde edilir (ekrandan, 02:30:00; python ile doğrulandı).

12.2 Örnek: $x(n) = x(n/3) + 1$

Bağıntı: $x(n) = x(n/3)+1$, $n > 1$, $x(1) = 1$. Böyle bir bağıntıyı çözmek için ikinci bir değişken atamak gerekir. Çözüm 3'ün tam kuvvetleri için yapılır: $n = 3^k$ (ekrandan, 02:30:00).

$x(n/3)$, $x(3^k/3^1) = x(3^{k-1})$ olarak yazılır:

$$x(3^k) = x(3^{k-1}) + 1$$

$x(3^{k-1}) = x(3^{k-2}) + 1$ yerine konur:

$$x(3^k) = x(3^{k-2}) + 1 + 1$$

Aynı şekilde $x(3^{k-2}) = x(3^{k-3}) + 1$ ve $x(3^{k-3}) = x(3^{k-4}) + 1$ ile:

$$x(3^k) = x(3^{k-3}) + 3 = x(3^{k-4}) + 4 = \cdots = x(3^{k-i}) + i$$

$x(1) = 1$ verildiği için $3^{k-i} = 1$ olmalıdır, yani $k - i = 0$, $i = k$:

$$x(3^k) = x(1) + k = k + 1$$

$n = 3^k$ olduğundan $k = \log_3 n$; sonuç $x(n) = \log_3 n + 1$'dir (python ile 3'ün kuvvetleri için doğrulandı).

Soru (öğrenci düzeltmesi): Bir öğrenci, açılımın ilk satırında $x(3^{k-1})$ yazılan yerde sol tarafın $x(3^k)$ olması gerektiğini söyledi. Hoca önce hangi yeri kastettiğini anlamadı, sonra öğrencinin haklı olduğunu kabul edip satırı $x(3^k) = x(3^{k-1}) + 1$ olarak düzeltti ve teşekkür etti.

12.3 Örnek: Küplerin Toplamı

Hoca ilk n sayının küplerinin toplamını hesaplayan bir algoritma yazdı: $S(n) = 1^3+2^3+\cdots+n^3$. Özyinelemeli sözde kod (ekrandan, 02:40:00):

```
Algorithm S(n):
    if n = 1 return 1
    else return S(n-1) + n*n*n
```

Bire gelindiğinde 1 döndürülüp çıkılır; değilse bir alt değer için çağrı yapılır ve her çağrıda o n'in küpü toplama eklenir.

İşlem sayısı için yineleme bağıntısı: $M(n)$, $M(n-1)$'i çağırır ve bunun dışında iki çarpma ($n \cdot n \cdot n$) yapar. Çıkış durumunda ($n = 1$) çarpma yapılmaz:

$$M(n) = M(n-1) + 2, \qquad M(1) = 0$$

Hoca bunun geriye doğru yerine koymayla çözümünü öğrencilere bıraktı ve sonucun $2n - 2$ olduğunu söyledi (ekrandan, 02:40:00). Aynı yöntemle $M(n) = M(n-i) + 2i$, $i = n-1$ ile $M(n) = 2(n-1) = 2n-2$ çıkar (ara adım eklendi; python ile doğrulandı).

Sonra hoca, aynı toplam özyinelemesiz (döngülü) yazılsaydı karmaşıklıkta bir fark olup olmayacağını sordu. Döngülü sürüm (ekrandan, 02:40:00):

```
for i = 2 to n
    sum = sum + i*i*i
return sum
```

Her adımda iki çarpma yapılır:

$$M(n) = \sum_{i=2}^{n} 2 = 2(n-2+1) = 2(n-1)$$

Karmaşıklık açısından iki sürüm arasında fark yoktur. Ama seçmek gerekirse döngülü sürüm tercih edilmelidir. Özyinelemeli sürümde çağrılardan gelen ekstra maliyet vardır: her özyinelemeli çağrı, ilgili parametrelerin sistem tarafından yığına atılmasını ve çekilmesini gerektirir. Karmaşıklık aynı görünse de döngülü sürüm daha mantıklıdır.

> **Ders sonrası not:** Tahtada döngülü sürümün sonucu $2(n-2+1) = 2n$ diye yazıldı (ekrandan, 02:40:00); doğrusu $2(n-1) = 2n-2$'dir ve özyinelemeli sürümle aynıdır. Asimptotik olarak ikisi de $\Theta(n)$'dir. Tahtadaki döngüde `sum` için başlangıç değeri görünmüyor; döngü 2'den başladığı için 1 ile başlatılması gerekir (ilk satır büyük olasılıkla ekranın dışında kaldı).

Örnek Sorular

Soru 1 (ekrandan, 00:10:00). A_1 2×3, A_2 3×4, A_3 4×2 boyutlu olsun. $(A_1A_2)A_3$ ve $A_1(A_2A_3)$ sıralarının skaler çarpma sayılarını hesaplayın. Hangisi daha iyidir?

> **Çözüm (derste verildi):** $(A_1A_2)A_3$: $2 \cdot 3 \cdot 4 + 2 \cdot 4 \cdot 2 = 24 + 16 = 40$. $A_1(A_2A_3)$: $3 \cdot 4 \cdot 2 + 2 \cdot 3 \cdot 2 = 24 + 12 = 36$. İkinci sıra daha iyidir.

Soru 2 (ekrandan, 00:21:40). Dört matrisin kaç farklı parantezlemesi vardır? Hepsini yazın.

> **Çözüm (derste verildi):** $m = n-1 = 3$ için $\frac{(2m)!}{(m+1)!\,m!} = \frac{6!}{4!\,3!} = 5$. Parantezlemeler: $A_1(A_2(A_3A_4))$, $A_1((A_2A_3)A_4)$, $(A_1A_2)(A_3A_4)$, $(A_1(A_2A_3))A_4$, $((A_1A_2)A_3)A_4$.

Soru 3 (ekrandan, 00:40:00). A_1 3×2, A_2 2×4, A_3 4×2, A_4 2×5 boyutludur. Dinamik programlamayla M ve P tablolarını doldurun ve en iyi parantezlemeyi bulun.

> **Çözüm (derste verildi, ara adımlar eklendi):** $d = (3, 2, 4, 2, 5)$. $M[1,2] = 24$, $M[2,3] = 16$, $M[3,4] = 40$, $M[1,3] = \min(28, 48) = 28$, $M[2,4] = \min(80, 36) = 36$, $M[1,4] = \min(66, 124, 58) = 58$. $P[1,2] = 1$, $P[2,3] = 2$, $P[3,4] = 3$, $P[1,3] = 1$, $P[2,4] = 3$, $P[1,4] = 3$. En iyi parantezleme $(A_1(A_2A_3))A_4$, en az 58 skaler çarpma.

Soru 4 (derste sözlü, 00:39:49). Dinamik programlamayla çözülen matris zinciri çarpımının zaman karmaşıklığı nedir?

> **Çözüm (derste verildi):** Tabloda yaklaşık $\frac{n(n+1)}{2} \approx n^2$ hücre doldurulur, her hücre için k en fazla n değer alır. Toplam $O(n^3)$.

Soru 5 (derste sözlü, 00:44:45). Matris zinciri çarpımı dinamik programlama (memoization) olmadan, doğrudan özyinelemeyle çözülseydi maliyet ne olurdu?

> **Çözüm (derste verilmedi):** Memoization yoksa aynı $M[i,j]$ değerleri ağacın farklı dallarında tekrar tekrar hesaplanır ve süre üstel olur. Bağıntının doğrudan özyinelemeli uygulamasında n matris için yapılan çağrı sayısı 3^{n-1}'dir ($n = 1, \ldots, 8$ için python ile sayıldı: 1, 3, 9, 27, 81, 243, 729, 2187). Yani $O(n^3)$ yerine üstel bir süre elde edilir.

Soru 6 (ekrandan, 00:50:25). 16 bitlik anahtar iki 8 bitlik parçaya bölünür: $h(key_1) = key_1 \bmod 7$ (alt parça), $h(key_2) = key_2 \bmod 11$ (üst parça), $h(key) = h(key_1) \,\|\, h(key_2)$. 2, 9 ve 16 anahtarları çakışır mı?

> **Çözüm (derste verildi):** Üçünün de üst parçası 0'dır, $h(key_2) = 0$. Alt parçalar $2 \bmod 7 = 9 \bmod 7 = 16 \bmod 7 = 2$. Hash değerleri aynıdır; üç anahtar çakışır.

Soru 7 (derste sözlü, 01:16:30). Yığın işlemlerine (PUSH, POP, MULTIPOP) aynı anda k eleman ekleyen MULTIPUSH eklenirse amortize maliyet hâlâ $O(1)$ olur mu?

Çözüm (derste verildi): Olmaz. MULTIPUSH her çağrıda k eleman ekleyebilir; her işlemin maliyeti $O(k)$ olur. Eklenen elemanlar ancak push edildikten sonra çekilebildiği için MULTIPOP'taki sınırlama MULTIPUSH için geçerli değildir.

Soru 8 (derste sözlü, 01:25:41). k bitlik ikili sayaçta INCREMENT'in yanında DECREMENT de olursa n işlemin en kötü durum maliyeti nedir?

Çözüm (derste verildi): En anlamlı biti 1, diğerleri 0 olan sayı azaltılınca k bit değişir; tekrar artırılınca yine k bit değişir. Bu ikisi art arda tekrarlanırsa her işlem k flip yapar; toplam $O(nk)$.

Soru 9 (ekrandan, 02:30:00). $x(n) = x(n-1) + 5$, $n > 1$, $x(1) = 0$ bağıntısını geriye doğru yerine koymayla çözün.

Çözüm (derste verildi): $x(n) = x(n-i)+5i$; $i = n-1$ ile $x(n) = x(1)+5(n-1) = 5n-5$.

Soru 10 (ekrandan, 02:30:00). $x(n) = x(n/3) + 1$, $n > 1$, $x(1) = 1$ bağıntısını $n = 3^k$ için çözün.

Çözüm (derste verildi): $x(3^k) = x(3^{k-i}) + i$; $i = k$ ile $x(n) = x(1) + k = k + 1 = \log_3 n + 1$.

Soru 11 (ekrandan, 02:40:00). $S(n) = 1^3 + 2^3 + \cdots + n^3$ değerini özyinelemeli hesaplayan algoritmanın çarpma sayısı için bir yineleme bağıntısı kurup çözün. Döngülü sürümle karşılaştırın.

Çözüm (derste verildi): $M(n) = M(n-1)+2$, $M(1) = 0$; çözüm $M(n) = 2n-2$. Döngülü sürüm de $\sum_{i=2}^{n} 2 = 2(n-1)$ çarpma yapar. Karmaşıklık aynıdır ($\Theta(n)$), ama döngülü sürüm özyinelemeli çağrıların yığın maliyeti olmadığı için tercih edilir.

Dersin Toparlaması

Dersin ilk bölümünde dinamik programlamaya matris zinciri çarpımı örneği eklendi. Çarpımın sırası sonucu değil maliyeti değiştirir; en iyi sıra, $M[i,j] = \min_{i \le k < j}(M[i,k] + M[k+1,j] + d_{i-1}d_k d_j)$ bağıntısıyla, maliyetleri tutan M ve ayrım noktalarını tutan P tabloları köşegenden yukarı doğru doldurularak bulunur. Olası parantezleme sayısı Catalan sayılarıyla üstel hızda büyürken dinamik programlama çözümü $O(n^3)$ sürer. Ardından önceki dersten kalan bir soruya, kısa ve düşük entropili anahtarların parçalı bir hash fonksiyonunda nasıl çakıştığını gösteren bir örnekle cevap verildi.

Dersin ana yeni konusu amortize analizdi. Birbiriyle bağlantılı işlemlerden oluşan bir dizide her işlemin en kötü durumunu toplamak fazla kötümserdir. Toplam analiz, muhasebe yöntemi ve potansiyel yöntemi aynı iki örnekte (MULTIPOP'lu yığın ve ikili sayaç) uygulandı; üçü de n işlem için $O(n)$, işlem başına $O(1)$ sonucunu verdi. İşlem kümesine MULTIPUSH ya da DECREMENT eklenince bu sonucun bozulduğu görüldü. Son olarak geriye doğru yerine koyma yöntemiyle üç yineleme bağıntısı çözüldü. Hoca bir sonraki derste örneklerle devam edeceğini, belki amortize analizle ilgili bir örnek daha yapacağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.